Compiscript compiler architecture

One core compiles source bytes into deterministic TAC and static code-generation metadata. CLI and HTTP are adapters; the IDE never implements another parser, semantic analyzer or TAC formatter.

Pipeline and ownership

```
.cps bytes
  -> frontend.Parse (ANTLR CST -> project AST + frontend diagnostics)
  -> semantic.Analyze (names/types/scopes + semantic diagnostics)
  -> tac.Generate (only when preceding diagnostics are empty)
  -> tac.BuildCodegenReport (only after successful generation)
  -> CompilationReport {tac, diagnostics, scopes, codegen?}
```

The entry point is Compile. Parse and semantic analysis each run once. Frontend/semantic diagnostics are stably ordered by span, phase, code and message, matching Analyze. Lowering errors reject the whole result: no partial TAC and no codegen report escape. Diagnostics carry phase/type code, message and source span; malformed/recovered ASTs and conservative initialization/scope violations can report `TAC_UNSUPPORTED` in `codegen`, even in unreachable source. No valid AST construction kind is rejected merely for being print, untyped or try/catch.

Source bytes, AST and semantic scope snapshots are inputs, not mutable codegen storage. Lowering derives private binding/layout metadata and the extended report joins semantic identities to emitted storage. Mixed acceptance checks source hashes, AST/scope snapshots, deterministic repeated Compile results and public facade parity.

Component	Responsibility / source
Frontend	frontend: lexer/parser diagnostics and project AST mapping, not lowering from generated CST types
Semantics	semantic: types, scopes, mutability, nominal declarations and static bounds
IR/lowering	tac: explicit instructions, temporary allocation, control transfers, calls, retained environments, objects/lists
Compile facade	compile.go: atomic pipeline and additive compilation report
Analyze facade	analyze.go: unchanged analysis-only report contract
CLI	compiscript-tac: text/JSON and exit policy; compiscript: legacy Analyze
HTTP / browser	server, web: one Compile call, server `Program.String()` as `tacText`, text-safe views
Execution proof	Test-only bounded frame/reference/environment oracles under compiler tests; no product VM

IR and storage at a glance

The TAC reference is authoritative for instruction fields and examples. Numeric operations explicitly widen integers with `i2f`; comparisons/not and branches model short-circuit conditions. Labels and jumps implement loops, boolean switch and nearest break/continue. Calls/returns use fresh frames; indirect calls use selected retained closures and signature guards. Object instructions allocate references, access fields and bind methods; list instructions allocate, read internal length and perform indexed loads/stores. Strings, functions, objects and lists are typed reference values, not numeric encodings.

T9 adds `print`, remainder, undefined-cell `declare`, lazy ternary joins, value-producing assignments and null-valued storage/calls. T10 adds checked unknown operations, dynamic member access/calls and synchronous handlers. This completes dispatch for the 18 statement and 14 expression kinds in ast/nodes.go; recovery `BadStmt`/`BadExpr` still reject.

The semantic `TypeError` absent-information marker remains unchanged; private guarded TAC carries actual runtime tags without adding `TypeAny`. An **unannotated field always retains an unknown storage contract**, even with a null/numeric initializer; no initializer leaves it undefined. Explicit

annotations still enforce widening, nominal identities, list shapes and full callable signatures before body effects. Missing function result annotations still mean null, not inferred return types. Catch bindings use exception-reference storage and can be captured.

Space / identity	Static meaning
`s<scopeId>.<name>`	Semantic binding identity, not a byte address
`globals`	Actually emitted non-frame/non-environment binding cells, including main block bindings
`main`	Emitted temporary cells; main has no invented activation frame
`frame:<procedure>`	Continuation, control/global-access links, optional result, receiver/ parameters, locals and temporaries
`environment:s<scopeId>`	Retained lexical binding layout and parent layout; each runtime entry is a distinct instance
`instance:<class>`	Inherited and own field offsets; methods map to procedure labels
Labels / temporary IDs	Control/procedure/layout labels are not cells; `t0...` cells are recycled after final consumption

`codegen` exposes semantic symbols, all actual locations, spaces/slots and labels. Offsets/sizes are **abstract cells**, not bytes or physical addresses. One reference cell does not describe heap payload size. A parameter/local can have both a frame staging slot and retained environment binding; inherited fields can appear in several instance layouts. An unemitted or non-storage symbol has no fabricated allocation. A static layout ID is never a live invocation/object address.

Practical choices and reasons

Choice	Reason and boundary
One pipeline and additive report	Avoid divergent compilers and preserve Analyze/TAC compatibility; derive layouts from actual emitted state rather than a second allocator
Smallest-free temporary reuse	Release after the final emitted consumer; bound demand across statements while retaining live values across effectful calls. Peak demand is not global optimal register allocation
Retained mutable lexical cells	Closures share binding identity, survive defining returns and separate different invocations; do not copy captured scalar snapshots
One call/frame mechanism	Bound methods and closures reuse invocation/result/guard machinery instead of inventing an object interpreter
Initializer policy A	Explicit user decision: all inherited/own fields initialize base-to-derived in declaration order, then only the nearest selected constructor runs once. No automatic ancestor constructor calls or undefined-field defaults; the regression observes 9
Fixed-length list references	Reuse semantic list types and preserve alias mutation; foreach freezes the initial reference/length, creates per-entry mutable bindings and uses the existing transfer stack
Strict HTTP decoding	Share Analyze's transport guards: valid JSON/UTF-8, exact unique `source`, no trailing data, 1 MiB bound and correct method/media type
Server-side TAC text	`Program.String()` serves CLI and HTTP; browser only renders results, avoiding duplicate formatting
Guard missing recovered loop block	Preserve existing located syntax recovery, not a blanket panic catch or invented empty body
Tagged unknown values, not new semantics	Reuse semantic identities and existing frames; validate actual operands and typed boundaries instead of changing Analyze or inventing defaults
Explicit handlers on the dynamic call stack	Preserve effects before a language fault, unwind to the nearest active handler and remove it before catch-body entry; lexical closures do not retain handlers

Choice	Reason and boundary
Check return before cleanup	Evaluate and type-check the return value while its handler is active; escaping return/break/continue remove only crossed handlers
Bounded print and fault categories	Deterministic text and explicit catchable language faults; malformed IR, resource budgets and host failures remain fatal, with no blanket recovery

Boundary of the result

Dynamic signature/initialization, null, field and list-bounds guards are backend obligations exercised by the test oracle, not stronger static Analyze promises. A dynamic-invalid program accepted by Analyze can generate guarded TAC and fault when executed in tests. Syntax/semantic errors still produce nil TAC/no codegen; conservative static initialization guards remain explicit. No assembler, physical ABI, garbage collector or product executor is claimed. The initial coverage gap is resolved, with no new concrete normative gap established here. Construction coverage is not exhaustive correctness or browser visual certification.